

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I C.C.Jyoshna (192411098)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C.c.Jyoshna

Q1. Case Study: SHA-256 Peak-Load Bottleneck

A pharmaceutical research lab uses **SHA-256 to hash every API transaction** so that the integrity of transaction data can be verified. During flash-sale events, the number of API requests increases suddenly, causing the hashing process to become a throughput bottleneck.

Analysis

The main problem is not the security of SHA-256 but the **large number of hashing operations occurring at the same time**. Processing every transaction sequentially increases CPU usage and response time during peak periods.

Proposed Solution

The lab can use a combination of **batching and hardware acceleration**:

- Transactions can be collected into small batches and processed together.
- Multiple CPU cores can calculate SHA-256 hashes in parallel.
- Hardware-supported SHA-256 acceleration can be used where available.
- A queue can temporarily hold incoming transactions during the peak period so that requests are not lost.

Example

Suppose **10,000 API transactions** arrive within a short period during a flash sale. Instead of processing all 10,000 transactions one by one, the system can divide them into batches and process several batches simultaneously using multiple CPU cores.

This reduces the processing delay and improves the number of transactions handled per second.

Integrity and Security

The optimization does **not replace or modify SHA-256**. Every individual transaction still receives its own SHA-256 hash. Therefore, the hash can still be used later to verify whether the transaction data has been changed.

Conclusion

The best approach is to use **batch processing with parallel CPU execution or hardware SHA-256 acceleration**. This removes the peak-load bottleneck while maintaining the same SHA-256 integrity guarantees.

Q2. Digital Signature Verification for an Altered Marksheet

Case Understanding

The telecom service provider digitally signs published exam results. A student claims that their marksheet was altered after signing. The objective is to determine whether the marksheet is the original signed document or whether its contents were modified later.

Application of Digital Signature Concepts

A digital signature provides three important security properties:

Authentication: The signature verifies that the marksheet was signed by the authorized institution.

Integrity: A hash of the marksheet is protected by the digital signature. If the marks are changed, the hash will no longer match.

Non-repudiation: The valid signature provides evidence that the authorized signer approved the document, assuming the private key was properly controlled.

Proposed Verification Process

1. **Collect the disputed marksheet and its digital signature.**
2. **Validate the digital certificate** used for signing by checking the trusted Certificate Authority, validity period, and revocation status.
3. **Verify the digital signature** using the public key in the certificate.
4. **Calculate the hash** of the disputed marksheet and compare it with the hash protected by the original signature.
5. **Check the trusted timestamp** to establish when the document was signed.
6. **Examine audit logs** for publication, signing, download, modification, or replacement activities.
7. Compare the disputed copy with the **original signed record** stored by the institution.

Case Analysis

For example, suppose the original signed marksheet contains **85 marks**, but the disputed copy contains **95 marks**.

If the hash of the disputed copy does not match the hash in the digital signature, signature verification will fail. This provides evidence that the document was modified after signing.

If the signature, certificate, timestamp, and audit records all validate successfully, the institution has stronger evidence that the document is the authentic signed version.

Solution

The organization should maintain:

- Digitally signed original marksheets.
- Trusted timestamps.
- Secure audit logs that cannot be easily modified.
- Proper protection of the institution's private signing key.
- Certificate revocation and renewal procedures.

Conclusion

Digital signatures establish **authentication, integrity, and non-repudiation**, while certificate validation, trusted timestamps, and audit logs provide additional evidence about **who signed the marksheet, when it was signed, and whether it was modified afterward**. This provides a reliable process for resolving the student's dispute.

Q3. Kerberos Replay Attack in a Smart-Metering IoT Platform

The smart-metering platform uses **Kerberos** to authenticate VPN users. Security monitoring has detected that a service ticket was reused after its expected validity period. This indicates a possible **replay attack**, where an attacker tries to reuse previously captured Kerberos authentication information.

How Kerberos Prevents Replay Attacks

Kerberos uses several mechanisms:

- **Timestamps:** The client sends a timestamp in the authenticator. The server checks whether it is recent and within the allowed clock-skew period.
- **Ticket lifetime:** Service tickets contain start and expiration times. A ticket should not be accepted after it expires.
- **Nonces:** Nonces are used in Kerberos exchanges to link responses to a particular request and help prevent reuse of old responses.
- **Authenticator checking:** The service verifies the authenticator and can detect a previously used authenticator.

Possible Misconfiguration

The breach could have occurred because of:

Excessive clock-skew tolerance: If the system allows a very large time difference between the client and server, an old authenticator may still be accepted.

Long ticket lifetime: If service tickets remain valid for too long, an attacker has a larger window in which a stolen ticket can be reused.

Disabled pre-authentication: This mainly weakens the initial authentication process and can make credential attacks easier. It is **not by itself the direct cause of a service-ticket replay**, but it can make account compromise more likely.

Poor replay-cache configuration: If previously used authenticators are not properly detected and rejected, replay attempts may succeed.

Example:

For example, suppose a service ticket should expire after a short period, but the VPN server is configured with an excessively long lifetime and large clock-skew tolerance. An attacker who obtains the ticket may be able to reuse it beyond the intended period.

The security team should therefore check **ticket lifetime settings, clock synchronization, replay-cache configuration, and pre-authentication settings** to identify the actual weakness.

Solution

The organization should:

- Keep client and server clocks synchronized.
- Use a small, appropriate clock-skew tolerance.
- Keep ticket lifetimes as short as practical.
- Ensure replay-cache protection is enabled.
- Enable Kerberos pre-authentication.
- Investigate and revoke potentially compromised tickets or credentials.

Conclusion

Kerberos prevents replay attacks through **timestamps, ticket lifetimes, authenticators, and replay detection**. In this case, excessive clock-skew tolerance, overly long ticket lifetimes, or inadequate replay detection could allow ticket reuse. Proper configuration and monitoring can significantly reduce the risk.

Q4. X.509 Certificate Trust and Lifecycle Management

The law firm uses **X.509 client certificates** to authenticate partner banks. A revoked certificate was still accepted, which indicates a failure in the certificate validation or revocation-checking process. This can allow an unauthorized party to access trusted systems.

Risks Involved

- **Impersonation:** An attacker possessing a revoked or compromised certificate may impersonate a legitimate partner bank.
- **Man-in-the-Middle (MITM):** If certificate validation is weak, an attacker may present an untrusted certificate and intercept communication.

- **Loss of trust:** Accepting invalid or revoked certificates weakens confidence in the entire certificate-based authentication system.
- **Expired certificates:** They may represent credentials that are no longer intended to be trusted.
- **Self-signed certificates:** If accepted without a controlled trust relationship, they may allow an attacker to create a certificate claiming to represent a trusted partner.

Proposed Certificate Lifecycle

The firm should establish a complete certificate lifecycle:

- **Certificate issuance:** Certificates should be issued only by approved Certificate Authorities (CAs).
- **Validation:** Check the certificate chain, validity period, issuer, subject, and intended usage.
- **Revocation checking:** Use **CRL or OCSP** to verify that certificates have not been revoked.
- **Renewal:** Replace certificates before they expire
- **Revocation:** Immediately revoke certificates when a private key is compromised or a partnership ends.
- **Trust-store management:** Only approved CA certificates should be present in the organization's trust store.
- **Monitoring and auditing:** Log certificate validation failures, renewals, revocations, and authentication attempts.

Example:

Suppose Partner Bank A's certificate is compromised and the CA revokes it. If the law firm's server checks the current revocation status, it will reject the certificate. If revocation checking is disabled or outdated, the revoked certificate may still be accepted, allowing an attacker to impersonate the bank.

Trust Management

Self-signed certificates should **not be accepted automatically**. If they are required for a specific controlled environment, they should be explicitly trusted and managed through a secure internal trust process.

Conclusion

The firm should enforce **certificate-chain validation, expiry checking, CRL/OCSP revocation checking, controlled trust stores, regular renewal, and continuous auditing**. This prevents revoked or untrusted certificates from being accepted and protects partner-bank communications from impersonation and MITM attacks.

Q5. HMAC-Based Authentication for Inter-Service Messages

Case Understanding

The pharmaceutical lab currently uses only a hash to check the integrity of messages. Since there is **no secret key**, an internal attacker can modify a message, calculate a new hash, and send the modified message with a valid-looking hash.

Why HMAC Prevents the Forgery

HMAC combines the message with a **secret key** before generating the authentication value.

An attacker may know the HMAC algorithm and see the message and HMAC, but without the secret key, they cannot generate a valid HMAC for a modified message.

Therefore, HMAC provides both:

Integrity: Detects changes to the message.

Authentication: Confirms that the message was created by a service that possesses the shared secret key.

Proposed HMAC Scheme

- Generate a strong random secret key for the communicating services.
- Store the key securely in a **key-management system or HSM**.
- Give the key only to the authorized services that need to communicate.
- For every message, the sender calculates an HMAC using the message and secret key.
- The sender sends the **message + HMAC** to the receiving service.
- The receiver uses its copy of the secret key to calculate the HMAC again.
- If both HMAC values match, the message is accepted; otherwise, it is rejected.

Example

Suppose Service A sends:

Patient_ID = 101, Result = Normal

Service A calculates an HMAC using the secret key and sends both the message and HMAC to Service B.

If an attacker changes the message to:

Patient_ID = 101, Result = Abnormal

the attacker cannot create the correct HMAC without knowing the secret key. Service B will calculate a different HMAC and reject the message.

Key Management

The lab should:

- Generate keys using a secure random generator.
- Store keys in a protected **HSM or key-management system**.
- Never include keys directly in source code.
- Restrict which services can access each key.
- Rotate keys periodically.
- Immediately replace a key if it is suspected to be compromised.
- Maintain secure records of key creation, rotation, and retirement.

Conclusion

A normal hash only detects changes; it does not prove who created the message because anyone can calculate the hash. **HMAC adds a secret key**, preventing an attacker without the key from creating a valid authentication value. Therefore, an HMAC-based scheme provides stronger message integrity and authentication for the lab's inter-service communication.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand